

Question 1

For $f_1(n) = n^2 + \log n + 75$. Terms: n^2 longn. constant 75.

As n becomes very large, n^2 dominates $n \log n$ and both dominate constant.

So dominant term is n^2

$f_1(n)$ is on the order of n^2 i.e. $f_1(n) \in O(n^2)$

For $f_2(n) = 2^n + n^2$. Terms: 2^n , n^2

Exponential functions grow faster than any polynomial when n is large

So dominant term is 2^n .

Therefore $f_2(n) \in O(2^n)$.

For $f_3(n) = n - n^{\frac{3}{4}}$ Terms: n and $n^{\frac{3}{4}}$

for large n , a higher power of n dominates a lower power.

So dominant term is n .

Therefore $f_3(n) \in O(n)$

$$2^n > n^2 > n. \quad \text{So } f_2 > f_1 > f_3.$$

Question 2

Algo 1

Step

operation

frequency

Initialization

3

1

while condition check

1

n

loop

4

n

final assignment

2

1

Total : $A_1(n) = 3 + (n+1) + 4n + 2 = 5n + 6$

Higher order term : $5n$ So ~~the~~ complexity is $O(n)$

Algorithm 2.

it has 2 nested loops.

initialization:

$Sum = 0$ $i = 0$. 2 operations $\rightarrow$ constant term

Nested loop. run n times for each i .

each iteration does

1 condition check. — 1 op

1 update. $Sum = Sum + a[i]$ — about 3 ops

1 increment — 1 op

around 5 operations per iteration, so $\approx 5n$ per run

Plus $j = 0 \rightarrow$ 1 constant

inner cost $\approx 5n + 1$

outer loop repeat n times. plus its own increment.

$$\text{total} \approx n \times (5n + 2) = 5n^2 + 2n$$

+ extra condition check $\approx 5n^2 + 3n$

$$\text{so } T_2(n) \approx 5n^2 + 3n + 2 + 1 = 5n^2 + 3n + 5$$

(ii) Algo 1: $T_1(n) = 5n + 6$

dominate term is $5n$

complexity is $O(n)$.

Algo 2: $T_2(n) = 5n^2 + 4n + 5$

dominate term is $5n^2$

so complexity is $O(n^2)$

(iii) $n^2 > n$

Algorithm 1 grows linearly with n while Algorithm 2 grows quadratically.

so Algorithm 1 is more efficient

Algorithm 2.

it has 2 nested loops.

initialization:

$Sum = 0$ $i = 0$. 2 operations $\rightarrow$ constant term

Nested loop. run n times for each i .

each iteration does

1 condition check. — 1 op

1 update. $Sum = Sum + a[i]$ — about 3 ops

1 increment — 1 op

around 5 operations per iteration, so $\approx 5n$ per run

Plus $j = 0 \rightarrow$ 1 constant

inner cost $\approx 5n + 1$

outer loop repeat n times. plus its own increment.

$$\text{total} \approx n \times (5n + 2) = 5n^2 + 2n$$

+ extra condition check $\approx 5n^2 + 3n$

$$\text{so } T_2(n) \approx 5n^2 + 3n + 2 + 1 = 5n^2 + 3n + 5$$

(ii) Algo 1: $T_1(n) = 5n + 6$

dominate term is $5n$

complexity is $O(n)$.

Algo 2: $T_2(n) = 5n^2 + 4n + 5$

dominate term is $5n^2$

so complexity is $O(n^2)$

(iii) $n^2 > n$

Algorithm 1 grows linearly with n while Algorithm 2 grows quadratically.

So Algorithm 1 is more efficient